

Storrflix: A Study On The Viability Of A Premium Decentralized Streaming Experience Using Torrents

Mouleeswaran K^{1†}, Mithran Kesavan^{1†} and Prakash
Murugan^{1†}

^{1*}Department of Computer Technology, Madras Institute of
Technology, Chrompet, Chennai, 600044, Tamil Nadu, India.

Contributing authors: mouleesh.kumar@gmail.com;
mithuk7b@gmail.com; prakash29112002@gmail.com;

[†]These authors contributed equally to this work.

Abstract

Multimedia is a form of communication that uses a combination of different content forms such as text, audio, images, animations, or video into a single interactive presentation, in contrast to traditional mass media, such as printed material or audio recordings. The release of Over-The-Top(OTT) apps revolutionized the multimedia industry, dealing a heavy blow to television and theatres. As time progresses and various new OTT apps get launched to compete in this market, users are forced to pay for each subscription individually. But more importantly, as more and more users starts streaming media through such services, maintaining centralised servers to support a big userbase has become increasingly difficult. An example of this is during the recent COVID pandemic, as OTT Platforms experienced a sudden exponential hike in number of users, they were forced to limit the maximum stream resolution to 720p and pushed even lower resolutions on mobile devices. The proposed solution is a decentralized media distribution system where-in each consumer becomes a node in a private tracker torrent swarm, thereby significantly reducing the load on the centralised media distribution servers, while also scaling extremely well with high user demand.

Keywords: Torrent, ElectronJS, Streaming, UDP, WebRTC, Networking, Media

1 Introduction

Entertainment media has become a crucial part of our society, as a means of escape and rest from our busy lives. It can be said that various streaming platforms are being used nowadays and most of them are not affordable by the general public. Even if affording one streaming platform started off as a cheap alternative to getting a TV connection or going to the movies, as time progressed, studios started making their specific streaming platform, and people are forced to buy various services to properly enjoy a streamlined experience. In recent years, 4K streaming has grown and is now one of the common resolutions to stream on. However, streaming 4K content requires a very high overhead on the centralized servers and handling a huge number of 4K streams simultaneously becomes increasingly expensive. This forces OTT Providers to charge more for their service. The increasing number of OTT services an average consumer needs to subscribe to, in addition to the exorbitant prices one has to pay to enjoy a streamlined media consumption experience has impacted the entertainment industry as a whole. A solution to this problem would be to replace the conventional centralized streaming servers with a decentralized torrent swarm made of private torrent trackers. Implementing this would result in less latency for getting streams of media, significantly less load on the primary servers of the OTT platforms, reduce costs, and almost eliminates downtime of the service in case of a server failure, as there is no central server in a decentralized system, since the data lost on one node can be recovered from other nodes quite easily.

2 Literature survey

- **Paper [1].** New improvements in web technologies enable more functionality that was previously only possible in natively developed apps. This opened up new opportunities to consolidate all development into only web apps, an app that runs in a web browser. This paper aims to understand the evolutions, capabilities, and limitations of developing web apps that run on any device. It presents a new concept of Progressive Web Apps, developed by Google, to normalize all web development. This demonstrates the main advantage of developing an app centrally as a Progressive Web App compared to developing the same solution for different mobile platforms and desktop platforms alike.
- **Paper [2].** The main purpose of web scraping is to extract information from one or more websites and process it into simple structures such as spreadsheets, databases, CSV files. However, web scraping is not only a very complicated task, it is also resource and time consuming if done manually. Previous studies have developed several automated solutions. The purpose of this article is to review various existing web scraping approaches, categories, and tools as well as their application areas.

- **Paper [3].** To develop accurate BT simulators and improve the performance of current BT systems, it is important to study the evolution of BitTorrent (BT) swarms on different timescales. In this paper, a hybrid measurement system is developed that includes both active and passive approaches. In addition to traditional resources, the DHT and PEX protocols allows the collection of more comprehensive information compared to previous measurement systems.
- **Paper [4].** The surge in traffic from peer-to-peer (P2P) applications, especially BitTorrent (BT), is putting unprecedented pressure on Internet Service Providers (ISPs). Therefore, P2P locales are widely proposed to alleviate costly ISP-to-ISP traffic. In this work, the existence and distribution of places are examined for the first time by large-scale hybrid PlanetLab-Internet measurements. Even in the most common Autonomous Systems (ASes), few single torrents can form a sufficiently large local cluster of peers, and modern single torrent locality mechanisms are highly inefficient. Inspired by the multiple torrent behavior of peers, a new framework was developed to track and recover content available on peers across multiple torrents, effectively extending local sharing capabilities.
- **Paper [5].** The purpose of this document is to understand and quantify the end-to-end latency observed by users of Over The Top (OTT) live TV services using Adaptive Bit Rate (ABR) technology. Analysis and testbed measurements show how key architectural elements (encoders, packagers, content delivery networks (CDNs), and players) contribute to this overall latency. Several improvements on architectural level and encoder implementation are proposed and partially evaluated through field experiments.
- **Paper [6].** The World Wide Web has become the world's premier source of information and communication. But at the same time, it has long been exploited as a platform for criminals to cooperate in various criminal activities. Over the past decade, law enforcement agencies have begun setting up organizational units dedicated to the task of scanning the entire Internet. Due to the enormous growth of information on the Web and limited human and time resources, this is a very difficult task. This paper applies current work on web mining systems to assist detectives in general scanning of suspicious or illegal websites. The system automatically analyzes websites, learns to classify websites, and filters suspicious and harmless web content so that it can distinguish between them automatically. Analysts no longer need to look at each URL individually from the list of suspicious websites. A simple but powerful learning technique known as the perceptron algorithm is applied to train the algorithm's class model.
- **Paper [7].** Popular content such as software updates are requested by a large number of users. Traditionally, large server farms or mirroring have

been used to meet high volumes of demand, both of which are expensive. A cost-effective alternative is a peer-to-peer based replication system in which the person retrieving the file simultaneously acts as client and server. In this paper, we'll explore BitTorrent, a new and already very popular peer-to-peer application that allows distribution of very large content to a large number of hosts. BitTorrent's analysis is based on measurements collected over a five-month period involving thousands of peers. The paper's conclusions make it clear that BitTorrent is a viable and cost-effective alternative to traditional server-based content delivery.

- **Paper [8].** With the words "Playing While Downloading", streaming media technology has become the most important transmission technology for online multimedia information, especially video information. The use of streaming media technology in data processing is changing the resource mode of online education.
- **Paper [9].** The launch of several movie streaming services has raised new questions about how online consumers deal with legal and illegal options for getting the products they want. This paper examines the factors that influence consumers' intention to subscribe to online movie streaming services. These services challenge the dramatic growth of illegal services in recent years. Based on the theory of planning behavior, existing models in the literature have been extended to include factors specific to consumer behavior in this area. Quantitative research was conducted on the Italian market and data analysis was performed using structural equation modeling. Attitude, product engagement, moral judgment, and frequency of past behavior were found to be the most important factors explaining intentions to pay for movie streaming services. This paper provides policy makers and industry managers with insight into the marketing communication strategies needed to minimize the risk of digital piracy.

3 Work flow of the proof of concept implementation

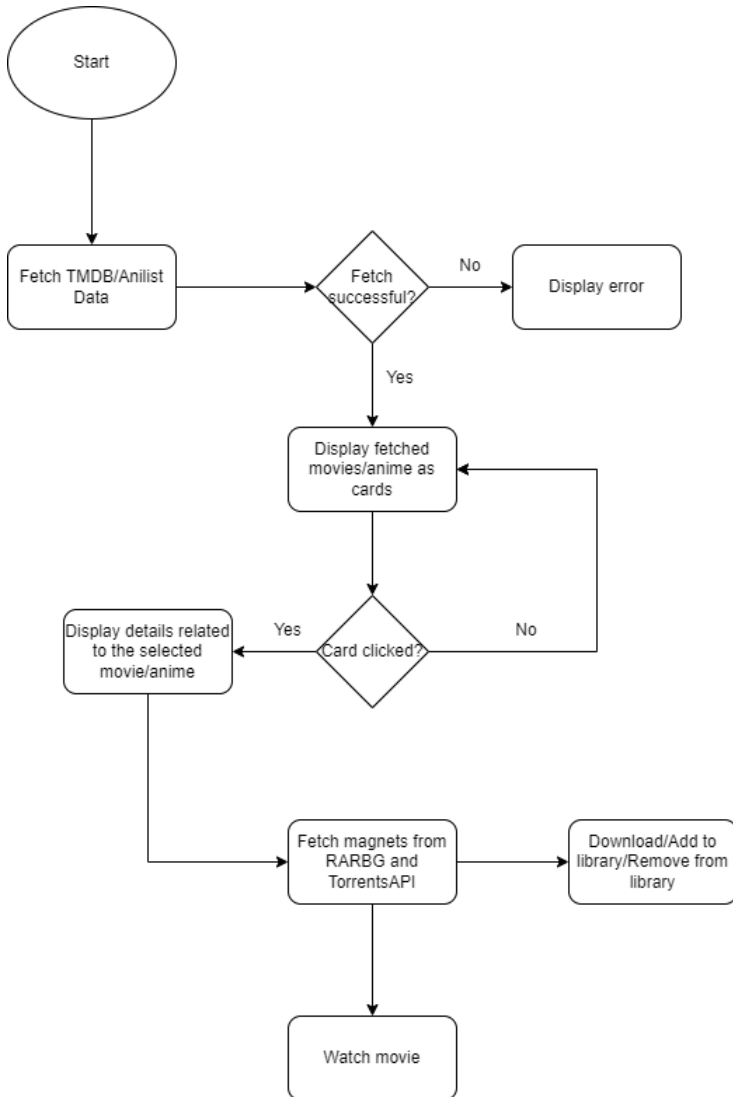


Fig. 1 Work flow of the proof of concept implementation

4 An Implementation Of The Proposed System

An example implementation of the proposed system has been developed using WebTorrent and Electron. While its entirely possible to maintain a BT swarm

using private torrent trackers, for the sake of demonstration, our implementation uses the public global BT swarm to obtain torrents for movies/series.

5 Webtorrent module to access the BT swarm

A Webtorrent module has been used to access the global BitTorrent swarm. WebTorrent is a streaming torrent client for node.js and the browser. It's written completely in JavaScript, so the same code works in both runtimes. In node.js, this module is a simple torrent client, using TCP and UDP to talk to other torrent clients.

In the browser, WebTorrent uses WebRTC (data channels) for peer-to-peer connection. It can be used without browser plugins, extensions, or installations. It requires only JavaScriptTM. It is also noted that WebTorrent does not support UDP/TCP peers in browser, this is a crucial need since most of the BT swarm is concentrated on these.

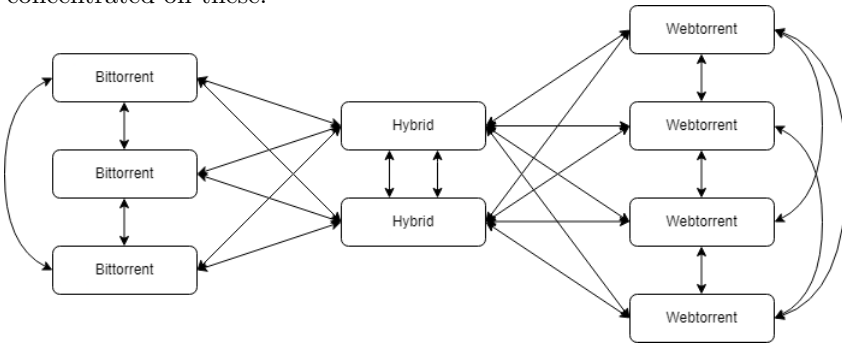


Fig. 4 Interaction between WebTorrent peers and BitTorrent peers

5.1 Working of Webtorrent.

Including the webtorrent.min.js script on our code starts fetching files over WebRTC using the BitTorrent protocol, or `require('webtorrent')` with browserify.

To make BitTorrent work over WebRTC (which is the only P2P transport that works on the web) some protocol changes were made. Therefore, a browser-based WebTorrent client or "web peer" can only connect to other clients that support WebTorrent/WebRTC.

Webtorrent can download multiple torrents simultaneously. It runs completely offline, hence fast. It supports various advanced torrent client features such as magnet uri and peer discovery.

Webtorrent has been used to implement the real time streaming and download capability in our app. It uses various API integration, such as Nyaa API and Rarbg API.

6 Electron JS

Electron is a run-time framework that is used to create desktop applications with web frontend frameworks such as React, while also utilizing the power of HTML5, CSS and JavaScript. The entire application is built on ElectronJS.

Any web application can run on Electron.js. Similarly, any Node.JS application can utilize this technology. Electron JS uses web technologies like simple HTML, CSS, and JavaScript. It can be designed for a single browser. Its file system that belongs to Node.js APIs and works on Linux, Mac OS X, Windows. It uses npm module. It consists of a native menu for dialogues and notifications. The windows installers do not need any configuration. It also has the facility of auto-updating and crash reporting on Windows and Mac with Squirrel. The crash reports are submitted to the remote server for further analysis. The content tracing activities like debugging and profiling are looked after by Chromium.

Electron.js creates a desktop application which further simplifies packaging, installation, updating, providing support for native menus, notifications, dialogues and in the end optimizing the app crash reports, Electron.JS takes care of all these essential steps.

Applications built with Electron are packaged into a particular version of Chromium and Node.JS, enabling the use of the features available in these versions in the application.

7 ReactJs

React (also known as React.js or ReactJS) is a free and open-source front-end JavaScript library[3] for building user interfaces based on UI components. It is maintained by Meta and a community of individual developers and companies. React can be used as a base in the development of single-page, mobile, or server-rendered applications with frameworks like Next.js. However, React is only concerned with state management and rendering that state to the DOM, so creating React applications usually requires the use of additional libraries for routing, as well as certain client-side functionality.

8 npm Modules

npm is the world's largest Software Registry. The registry contains over 800,000 code packages. Open-source developers use npm to share software. Many organizations also use npm to manage private development. npm is also free to use.

One can download all npm public software packages without any registration or login. npm includes a CLI (Command Line Client) that can be used to download and install software.

The name npm (Node Package Manager) stems from when npm first was created as a package manager for Node.js. All npm packages are defined in a file called package.json. The content of package.json must be written in JSON. At least two fields must be present in the definition file: name and version.

```
{
  "name": "mmdm",
  "version": "1.0.0",
  "author": "joemama",
  "main": "src/index.js",
  "scripts": {
    "start": "SET NODE_ENV=development & concurrently \"\n",
    "web:watch": "vite",
    "electron:start": "electron src",
    "build": "vite build && electron-builder",
    "publish": "vite build && electron-builder -p always
  },
  "devDependencies": {
    "builtin-modules": "^3.2.0",
    "concurrently": "^7.0.0",
    "electron": "^18.2.1",
    "electron-builder": "^22.14.13",
    "electron-notarize": "^1.1.1",
    "less": "^4.1.2",
    "react": "^18.1.0",
    "react-dom": "^18.1.0",
    "sass": "^1.52.1",
    "vite": "^2.8.6",
    "vite-plugin-commonjs-externals": "^0.1.1"
  },
}
```

Fig. 3 Node modules used in the application.

The modules on which the application depend on are called as 'Dependencies'. Various dependencies have been used to make the application as efficient and user friendly as possible.

The node modules/dependencies used in the project are:

Technology	Function
splidejs/react-splide	React Splide is a React component for a Splide slider/carousel
vitejs/plugin-react	Vite.js allows developers to set up a development environment for frameworks like Vue and React and even for Vanilla JavaScript app with a dev server. Moreover, it allows the development team to hot reload in just three commands.
anitomyscript	Pure javascript bindings for Anitomy, ported with emscripten.
antd	A CSS library that contains a set of high quality components and demos for building rich, interactive user interfaces. It is the second most popular CSS library in the world.

axios	Axios is a promise-based HTTP Client for node.js and the browser. It is isomorphic, i.e, it can run in the browser and nodejs with the same codebase. On the server-side it uses the native node.js http module, while on the client it uses XMLHttpRequests.
bootstrap	A powerful, extensible, and feature-packed frontend toolkit. It is the most popular CSS library in the world.
electron-updater	This module allows you to automatically update your application. You only need to install this module and write two lines of code. To publish updates, simple file hosting alone is required and does not require a dedicated server.
matroska-subtitles	Streaming parser for embedded .mkv subtitles.
query-string	Parse and stringify URL query strings
rarbg-api	This is a nodejs api wrapper for rarbg.to website.
react-bootstrap	React-Bootstrap replaces the Bootstrap JavaScript. Each component has been built from scratch as a true React component, without unneeded dependencies like jQuery.
react-icons	Include popular icons in React projects easily and utilizes ES6 imports that allows to include only the icons that the application is using.
react-native-rss-parser	React Native compatible RSS parser.
react-player	A React component for playing a variety of URLs, including file paths, YouTube, Facebook, Twitch, etc.
react-router-dom	It contains bindings for using React Router in web applications.
screenful	Simple wrapper for cross-browser usage of the JavaScript Fullscreen API, which lets us bring the page or any element into fullscreen. It smoothenes out the browser implementation differences.
styled-components	Utilising tagged template literals (a recent addition to JavaScript) and the power of CSS, styled-components allows you to write actual CSS code.
webtorrent	WebTorrent is a streaming torrent client for node.js and the browser. In node.js, this module is a simple torrent client, using TCP and UDP to talk to other torrent clients.

9 API Integration

The modern web development world is impossible to imagine without JavaScript. Over the years of its growth, this language has gone from a small add-on to a multi-functional and powerful tool. Today JavaScript successfully helps developers with both front-end and back-end work. Often the application functionality is mainly related to interaction with various APIs.

API (Application Programming Interface) can be considered as a set of rules that are shared by a particular service. These rules determine in which format and with which command set your application can access the service, as well as what data this service can return in a response. An API acts as a layer between your application and external service. All browsers have a set of built-in Web APIs to support complex operations, and to help accessing data. For example, the Geolocation API can return the coordinates of where the browser is located.

Mutiple third party APIs for different functions are used:

- AniList API has been used to display the poster, thumbnail, description, characters, trailers and genre of anime.
- Tmdb API has been used to display the poster, description, casts and trailers of movies on the app.
- The Tmdb API also displays the poster, description, casts and trailers of series on the app.
- Rarbg API for torrenting and streaming movies and series on the app.
- Nyaa API has been used for torrenting and streaming of Anime series and Anime Movies.

To make API calls using JavaScript, a reference can be made under the 'script' tag to the JavaScript library which contains functions and other configuration parameters pertaining to the API should be made. A good API always maintains appropriate documentation to its functions and parameters. In most cases, this js library is made available via the Content Delivery Network(CDN) in order to be responsive. JavaScript contains functions for serializing and de-serializing a JSON object and thus handling JSON responses and traversing/parsing the response is also managed within the same script.

10 React Player

React Player is a React component for playing a variety of URLs, including file paths, YouTube, Facebook, Twitch, SoundCloud, Streamable, Vimeo, Wistia, Mixcloud, DailyMotion and Kaltura. The component parses a URL and loads in the appropriate markup and external SDKs to play media from various sources. Props can be passed in to control playback and react to events such as buffering or media ending. For platforms without direct use of npm modules, a minified version of ReactPlayer is located in "dist" directory after installing, which is generated using build:dist.

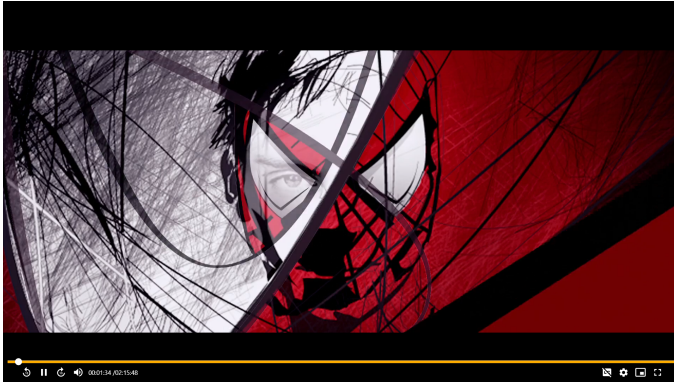


Fig. 5 React Player used in our project

11 Features of Application

11.1 Easily Navigable GUI

- A side bar which can be opened by clicking the hamburger menu has been created for easy navigation through the different media available.

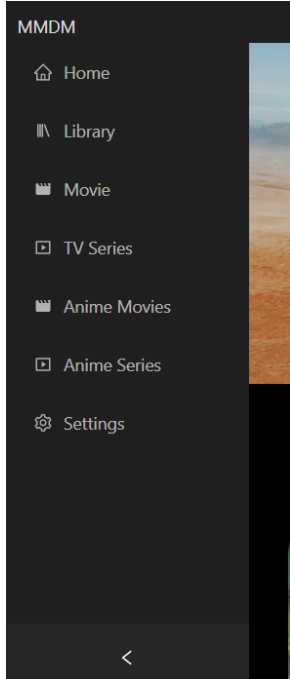


Fig. 6 Menu Open

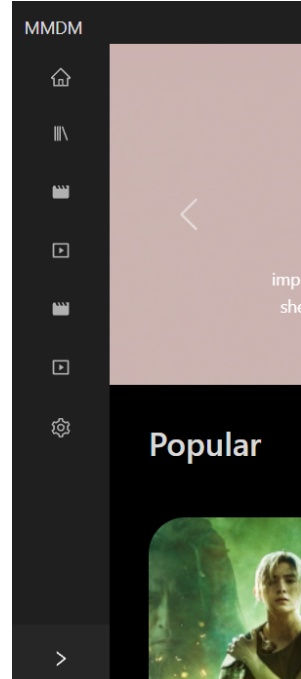


Fig. 7 Menu Close

- A search bar has created, which can be used to search for different media by typing in a part of its title.
- The thumbnail of every media is shown along with its title in the home page.

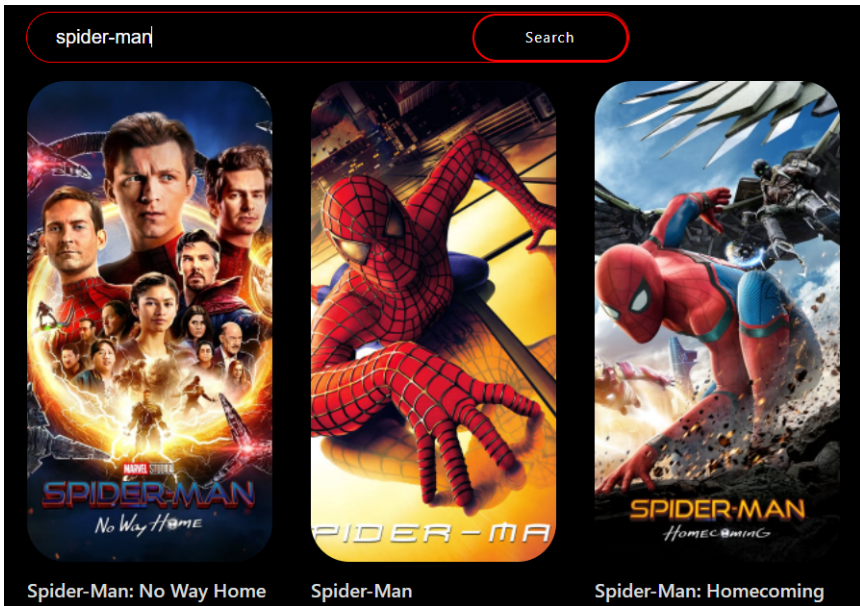


Fig. 8 Searching for Title

- When a specific media is chosen, the poster, cast(only movie, series), and trailer will be shown and can be viewed.

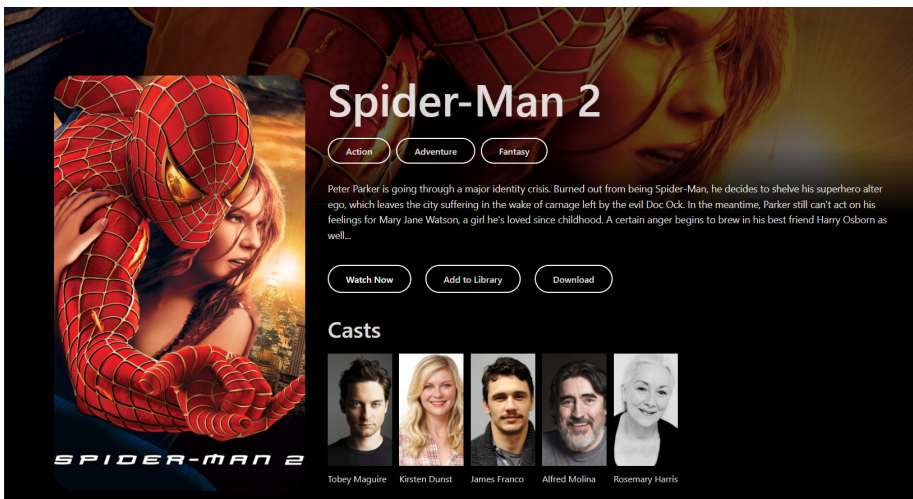


Fig. 9 The Description Page

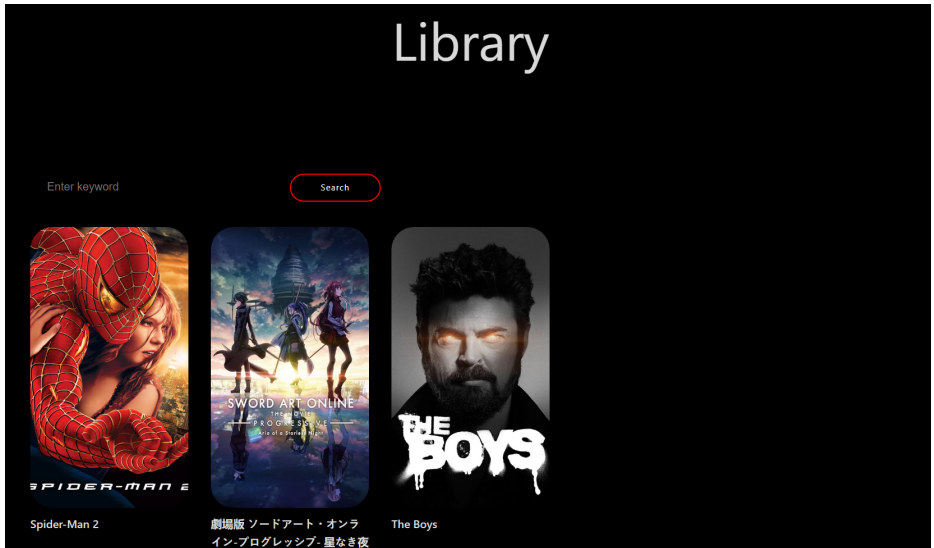


Fig. 10 Library with search function

- The trailers are an embedded YouTube link, which can be played through the app rather than getting redirected to the website.
- A library to save all the media that you want to continue following or watch later.
- The app works on any Operating System as it is a WebApp

11.2 Streaming and Downloading

Any media available on the app (Movies, Series and Anime) can be streamed natively on the application through the use of web-torrent. The application has a custom built player which can pause, forward, reverse and can also navigate through the media using the slider. The media can also be downloaded rather than streaming for viewing it offline.

11.3 Timeline

A timeline which saves the history of the media watched has been implemented. Media is sorted according to the dates they were viewed on. When the user wants to re-watch a media, they can use the timeline to backtrack and search for it. When the media is clicked on, it continues from the last viewed time stamp.

11.4 My Library

The media that are favoured by the user can be added to their library. To add a media to a library, the user has to 'follow' the media. The media then gets added to the library and can even be accessed through it. The user is regularly notified whenever the media gets updated.

12 Security Concerns

Since our application fully follows the safety protocols that are in compliance with other popular applications' safety standard, it is very safe from viruses and other harmful malware attacks. All of our dependencies are open-source.

13 Result

Mainstream OTT platforms adhere to the illusory compulsion of hosting centralized servers. Switching to a decentralized infrastructure can only be beneficial for all the involved parties as it solves most of the problems that come with such streaming services today such as low scalability, high costs, subpar resilience to network outages, and privacy concerns. Storrflix serves as a proof of concept for this methodology as it relies entirely on torrent swarms for streaming, without requiring the use of any moderated servers. As opposed to centralized server management, this approach is furthermore welcoming towards any influx in the user base, as they themselves are able to handle much of the incoming traffic.

References

- [1] D. Fortunato and J. Bernardino, "Progressive web apps: An alternative to the native mobile Apps," 2018 13th Iberian Conference on Information Systems and Technologies (CISTI), 2018, pp. 1-6, doi: 10.23919/CISTI.2018.8399228.
- [2] R. Diouf, E. N. Sarr, O. Sall, B. Birregah, M. Bouso and S. N. Mbaye, "Web Scraping: State-of-the-Art and Areas of Application," 2019 IEEE International Conference on Big Data (Big Data), 2019, pp. 6040-6042, doi: 10.1109/BigData47090.2019.9005594.
- [3] M. Su, H. Zhang, B. Fang and N. Yan, "A measurement study on swarm evolution of BitTorrent," 2012 International Conference on Cyber-Enabled Distributed Computing and Knowledge Discovery, 2012, pp. 512-518, doi: 10.1109/CyberC.2012.94.
- [4] H. Wang and J. Liu, "Exploring Peer-to-Peer Locality in Multiple Torrent Environment," in IEEE Transactions on Parallel and Distributed Systems, vol. 23, no. 7, pp. 1216-1226, July 2012, doi: 10.1109/TPDS.2011.253.

- [5] Sridhar, V. (2019). Over-the-Top (OTT) Services—Should They Be Regulated Much like Telecom Services?. In: Emerging ICT Policies and Regulations. Springer, Singapore. <https://doi.org/10.1007/978-981-32-9022-8>
- [6] Coote, Ravi and Rein, Kellyn and Schade, Ulrich. (2013). Search and Detection of Illegal Web Content.
- [7] Izal, M., Urvoy-Keller, G., Biersack, E.W., Felber, P.A., Al Hamra, A., Garcés-Erice, L. (2004). Dissecting BitTorrent: Five Months in a Torrent's Lifetime. In: Barakat, C., Pratt, I. (eds) Passive and Active Network Measurement. PAM 2004. Lecture Notes in Computer Science, vol 3015. Springer, Berlin, Heidelberg, doi: 10.1007/978-3-540-24668-8-1
- [8] Wang, T. (2012). The Construction of Online Video Resource Library with Streaming Media. In: Tan, H. (eds) Knowledge Discovery and Data Mining. Advances in Intelligent and Soft Computing, vol 135. Springer, Berlin, Heidelberg. doi: 10.1007/978-3-642-27708-5-97
- [9] Sardanelli, D., Vollero, A., Siano, A. et al. Lowering the pirate flag: a TPB study of the factors influencing the intention to pay for movie streaming services. Electron Commer Res 19, doi: 10.1007/s10660-019-09346-7